

MATLAB Notes

Official Quick Reference -

<https://matlabacademy.mathworks.com/artifacts/quick-reference.html?course=gettingstarted&language=en&release=v1>

Commands:

- ◆ `clear all` - Clears the workspace (where all variables are shown)
- ◆ `clc all` - Only clears the command panel ensuring that the variables stay in the workspace panel
- ◆ `close all` - Closes all the opened figures, plots, etc.
- ◆ `save datafile` - Saves your MATLAB file with **datafile** as the file name (MATLAB files are stored with *.mat* extension)
- ◆ `load datafile` - Loads your datafile.mat file
- ◆ To load or save only *some* of your variables, you can use additional inputs with the commands. For example, The provided file `myData.mat` contains multiple variables. Try loading just the variable `k`.

```
load myData k
```

- ◆ Then try saving the variable `k` to a new MAT-file named `justk.mat`.

```
save justk k
```

- ◆ **Some Built-in Constants & Functions:**

1. `pi`
2. `sin()`
3. `cos()`
4. `eig()` (for eigen values)
5. `abs()` (for absolute values)
6. `sqrt()` (for squareroot calculation)

```
7. i (for imaginary number 'iota')
8. sqrt(-9) gives 0.0000 + 3.0000i !
9. round() [Rounds off]
10. randi() [https://in.mathworks.com/help/matlab/ref/randi.html]
```

◆ In MATLAB, the `format` function is used to control the display of decimal places. For example, `format long` displays numbers with more decimal places, while `format short` displays numbers with fewer decimal places.

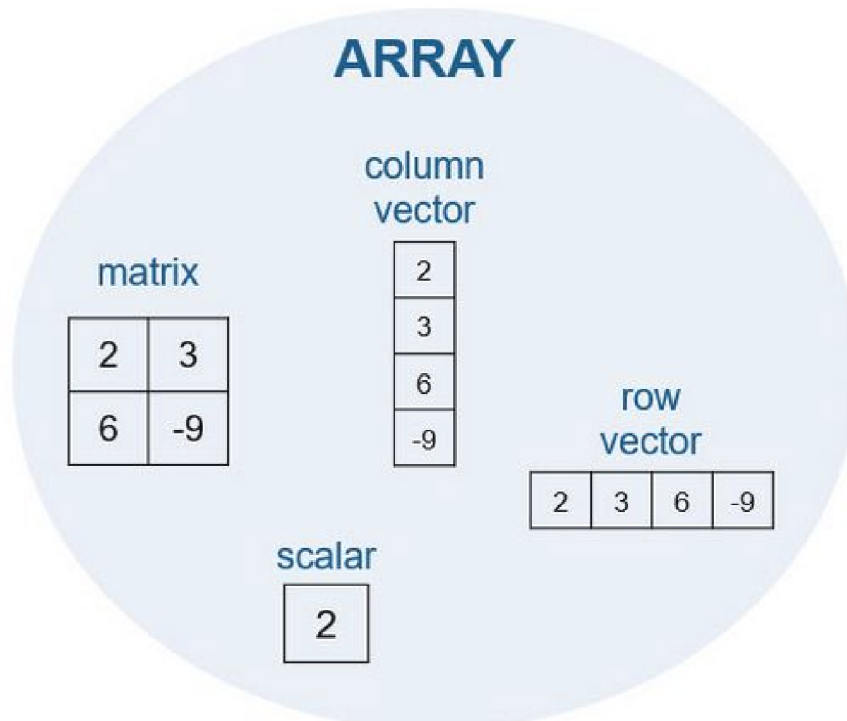
Arrays

1. Arrays are - Matrix, Row Vector, Column Vector, Scalar

What's an Array?

All MATLAB variables are *arrays*. This means that each numeric variable can contain multiple numbers. You can use arrays to store related data in one variable.

Because you'll use arrays every time you program, it's important to get to know them and the terminology used to describe them.



Example creations

Task 1	<code>x = 4</code>	<code>x = 4</code>
Task 2	<code>y = [7 9]</code>	<code>y = 1×2</code> 7 9
Task 3	<code>z = [7; 9]</code>	<code>z = 2×1</code> 7 9
Task 4	<code>n = [3, 10, 5]</code>	<code>a = 1×3</code> 3 10 5
Task 5	<code>b = [8; 2; -4]</code>	<code>b = 3×1</code> 8 2 -4
Task 6	<code>c = [5 6 7; 8 9 10]</code>	<code>c = 2×3</code> 5 6 7 8 9 10
Task 7	<code>d = [sqrt(10), pi^2]</code>	<code>d = 1×2</code> 3.1623 9.8696

Fact:

Arrays are a common data structure used throughout MATLAB. In fact, MATLAB is an abbreviation for MATrix LABoratory. Most MATLAB functionality can work on multiple values at once.

MATLAB provides some flexibility for creating arrays. For example, these commands are all valid ways to create the same array.

```
x = [7 9]
x=[7,9]
x = [7, 9]
```

2. : Operator

```
> x = [1, 2, 3, 4, 5]
> x
```

```
1 2 3 4 5
```

```
> y = [1:5]
> y
```

```
1 2 3 4 5
```

3. Intervals

Example 1:

```
> x = [20:2:26]
> x

20 22 24 26    % Prints 20 to 26 in intervals of 2
```

Example 2:

```
> y = [1:0.5:5]
> y

1.0000  1.5000  2.0000  2.5000  3.0000  3.5000  4.0000  4.5000  5.0000
```

4. linspace(first, last, number_of_elements)

Example:

```
> x = linspace(0,1,5);
> x

0  0.250  0.500  0.750  1.000
```

5. Transpose Operator: `'`

Example:

```
> x = [1:3]
> x

1 2 3

> x = x'
> x

1
2
3
```

6. Combine them all

Example:

```
> x = (1:2:5)'
> x
```

```
1  
3  
5
```

1. `rand(2)` creates a 2x2 matrix of random numbers less than 1
2. `rand(3,4)` creates a 3x4 matrix of random numbers less than 1
3. `ones(2)` creates a 2x2 matrix of only ones '1'
4. `ones(4, 6)` creates a 4x6 matrix of only ones '1'
5. For only zeros '0', use `zeros()` function.
6. `size()` operator

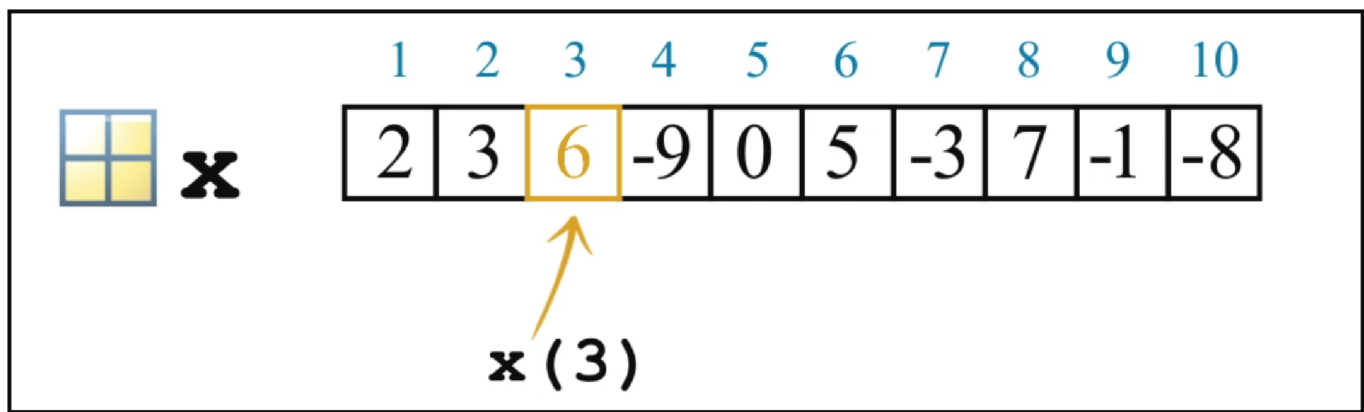
Example:

```
> x = rand(2, 5)  
> size(x)  
  
2  5  
  
> y = rand(6)  
> size(y)  
  
6  6
```

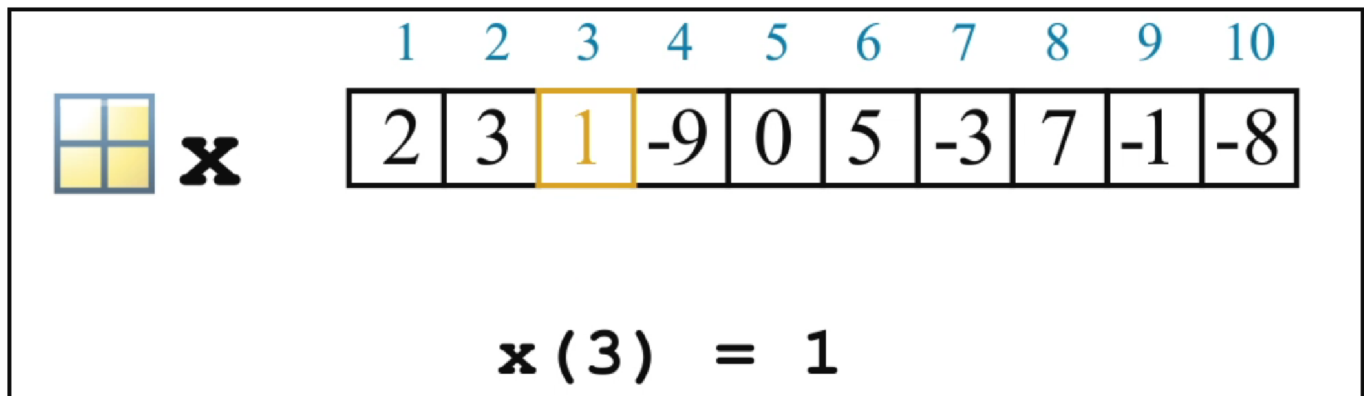
7. Combined 🚀

```
> rand(size(x)) % Creates a random 2x5 matrix
```

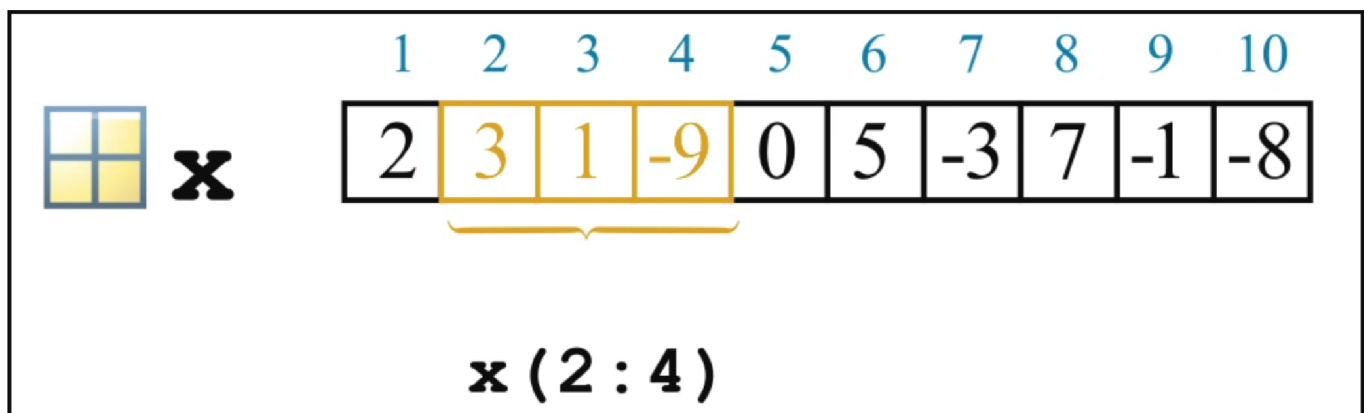
Matrix Indexing



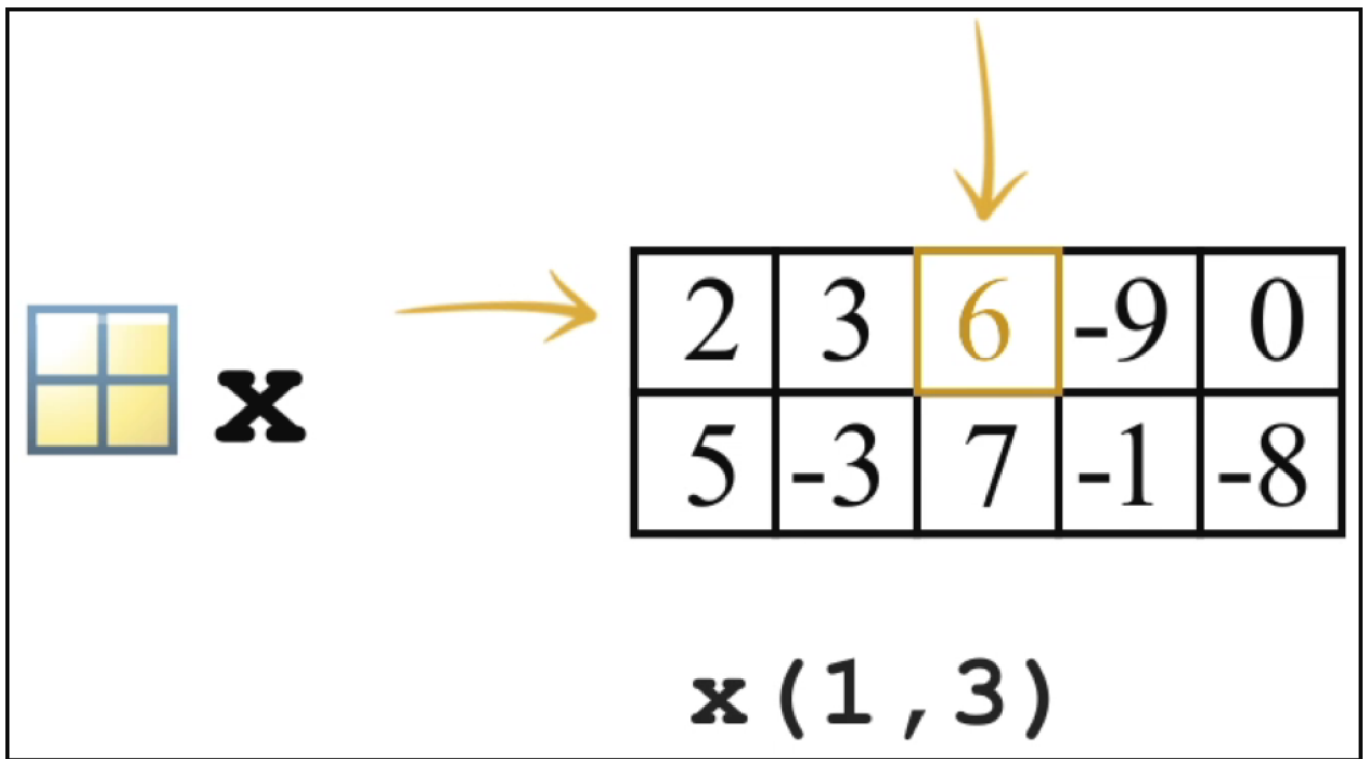
Indexing in vectors starts from 1 and not 0!



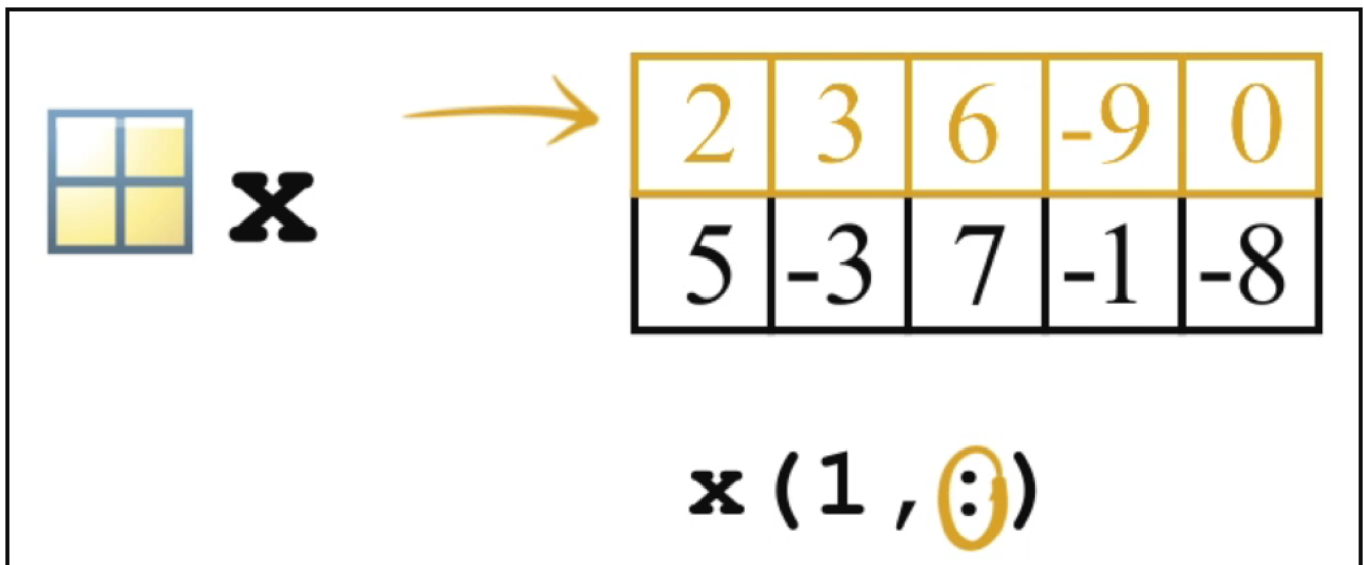
You can manipulate the values of the vector like this



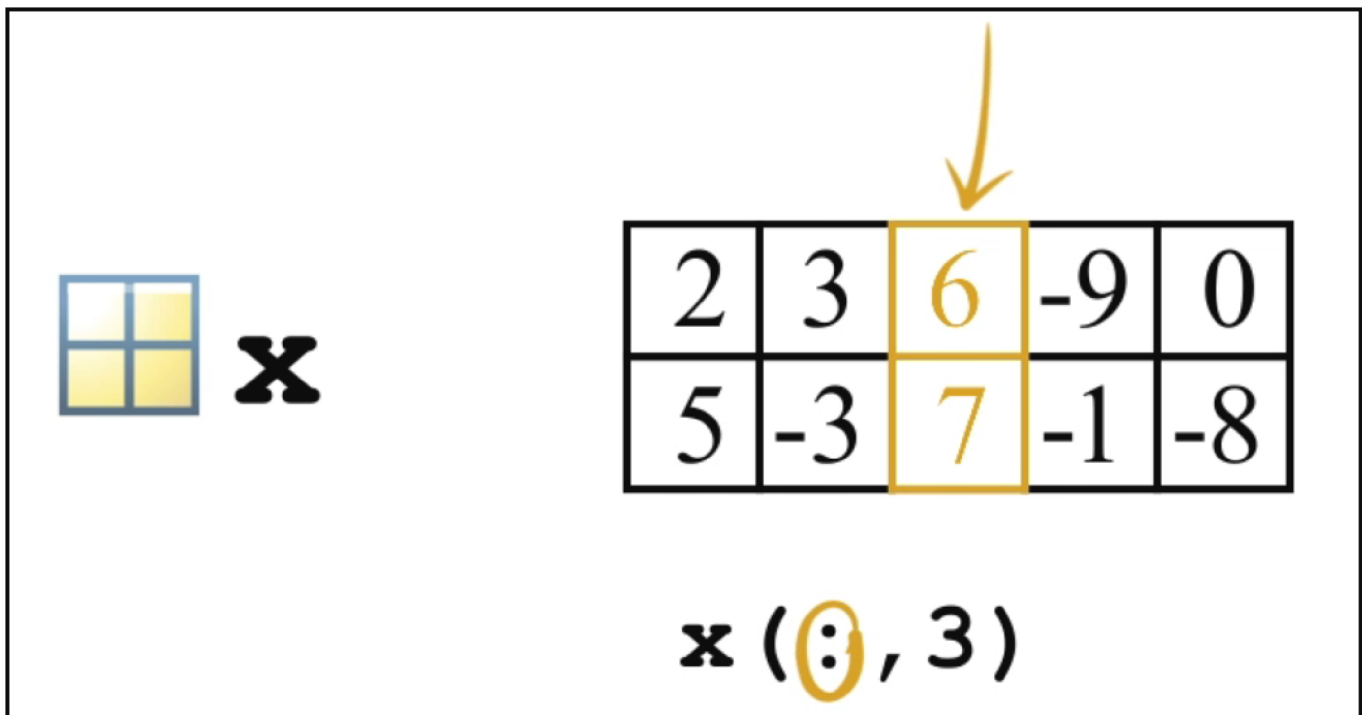
Selecting multiple indexes



You can choose a position like this from a 2D matrix



Using : at column's position selects all columns in 1st row!



Selecting all rows in 3rd column

2. end Keyword

Example shown:

```
load datafile
data
```

Task 1

```
x = data(6, 3)
```

Task 2

```
y = data(end, 3)
```

Task 3

```
z = data(end-1, 3)
```

Further Practice

```
idx = 8;
w = data(idx)
```

```
data = 7x4
    3.0000    0.5300    4.0753    NaN
   18.0000    1.7800    6.6678    2.1328
   19.0000    0.8600    1.5177    3.6852
   20.0000    1.6000    3.6375    8.5389
   21.0000    3.0000    4.7243   10.1570
   23.0000    6.1100    9.0698    2.8739
   38.0000    2.5400    5.3002    4.4508
```

```
x = 9.0698
```

```
y = 5.3002
```

```
z = 9.0698
```

```
w = 0.5300
```

end refers to the last element of the row/column

3. You can use arithmetic with the keyword end . For example:

```
> y = data(end-1, end-2)
```


4. You can use the colon operator to specify a range of values. This code creates a matrix containing the first, second, and third rows of the matrix `A`.

```
> x = A(1:3, :)
```

5. To index into a vector, use a single index value. For example, this code returns the third element of row or column vector `v`.

```
> x = v(3)
```

6. You can use a single range of index values to reference multiple vector elements. For example, this code returns a subset of vector `v` containing the elements from index `3` to the end.

```
> x = v(3:end)
```

7. Indices can be nonconsecutive numbers. For example, to extract the first, third, and sixth elements of `density`, use `[1 3 6]` as an index.

```
> density

0.5300
1.7800
0.8600
1.6000
3.0000
6.1100
2.5400

> fun = density([1, 3, 6])
> fun

0.5300
0.8600
6.1100
```

8. You can change the elements in a vector by combining indexing with assignment. This code changes the second element of `A` to 11.

```
> A(2) = 11
```

```
> % Changes the value of element at 1st row and last column to 0.5
```

```
> data(1, end) = 0.5
```

9. Combine indexing with assignment to change array elements to other elements.

```
> % For vectors
```

```
> x(1) = x(2)
```

```
> % Changes the value of first column of data to the second column of data
```

```
> data(:, 1) = data(:, 2)
```

10. You can do scalar operations on array, for example:

```
> % You can add a scalar value to all the elements of an array
```

```
> x = [1 2 3];
```

```
> y = x + 2
```

```
> y
```

```
3 4 5
```

```
> % You can add any two arrays of the same size.
```

```
> z = x + y
```

```
> % You can multiply/ divide all the elements of an array by a scalar
```

```
> z = 2*x
```

```
> y = x/3
```

11. You can apply basic statistical functions in MATLAB to a vector to produce a single output. For example, the maximum value of a **vector** can be determined using the `max` function.

```
> xMax = max(x)
```

12. MATLAB has functions that perform mathematical operations on an entire array of values in a single command.

```
> % You can find the square root of each element in array 'x' with this syntax.  
> xSqrt = sqrt(x)  
  
> % You can round off all the elements of an array using round() function.  
> xRound = round(x)
```

MATRIX MULTIPLICATION

The `*` operator performs [matrix multiplication](#). So, if you use `*` to multiply two equally sized vectors, you get an error message because the inner dimensions do not agree.

```
z = [3 4] * [10 20]
```

```
Error using *  
Incorrect dimensions for matrix  
multiplication.
```

The `.*` operator performs element-wise multiplication by multiplying the corresponding elements of two equally sized arrays.

```
z = [3 4] .* [10 20]
```

```
z =  
    30    80
```

You can only use `*` when you either multiply a matrix by a **scalar** value or when you want to multiply a $P \times Q$ matrix with $Q \times R$ matrix

 **Example (DIY)**

```
> x = [1 2; 3 4; 5 6; 7 8].*[1; 2; 3; 4]
```

FUNCTION CALLS

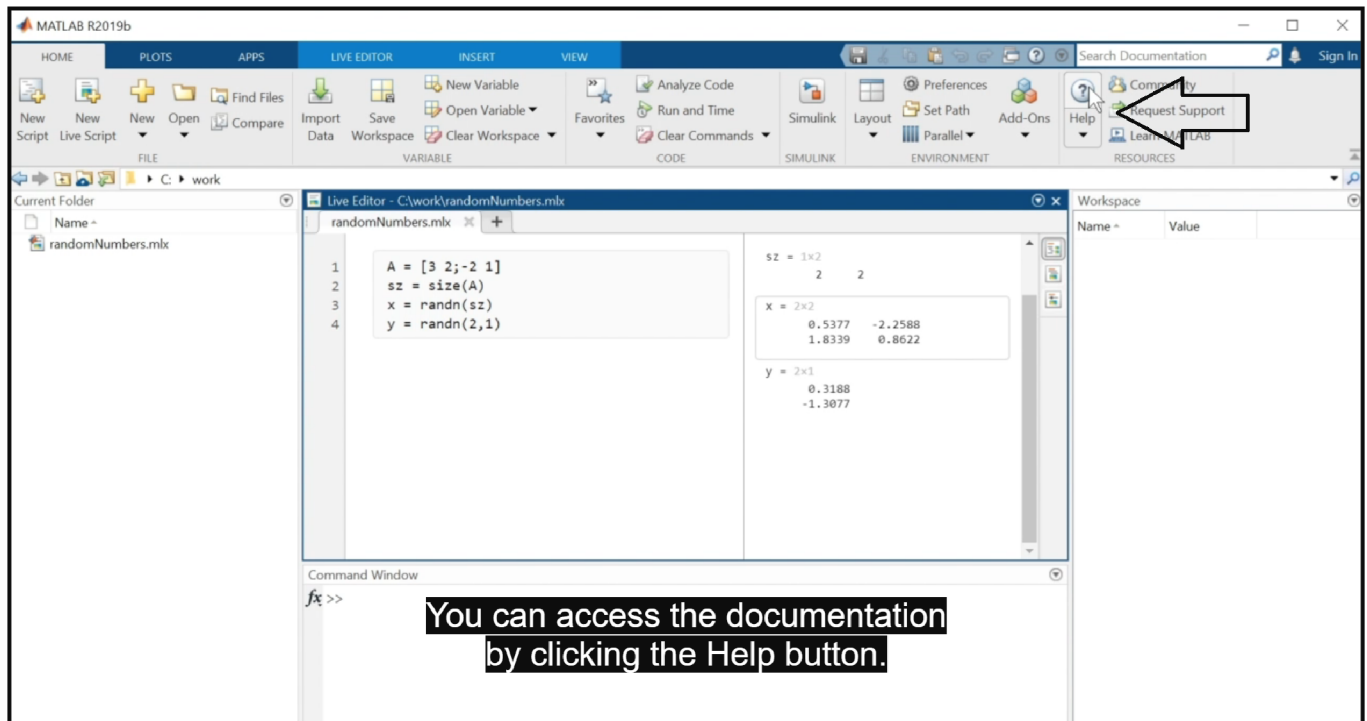
Understand w/ Examples:

```
> % Stores the size of the vector in a row-vector format in s
> s = size(x)

> % Stores the size of row in xrow and size of column in xcol
> [xrow, xcol] = size(x)

> % Here, xMax stores the max value and idx stores the index of max value in x
> [xMax, idx] = max(x)
```

Use MATLAB Documentation



Here, you can search about any function that you aren't aware of in any code if you stumble upon!

Example: `randi()` function

Go through the documentations: <https://in.mathworks.com/help/matlab/ref/rand.html>

Tips

You can also open the documentation using the `doc` function. If you know the name of the function and want more information about it, you can use this command: `doc randi`
However, if you don't know the name of the function, you can search the documentation using phrases. Try searching the documentation for a function that creates normally distributed numbers (instead of uniformly distributed numbers) using: `doc normally distributed numbers`

Then we dive deep into plotting and programming using MATLAB which includes for loop and if-else

Useful Links for Plotting:

Links:

1. Plot Gallery (Can also download codes):
<https://in.mathworks.com/products/matlab/plot-gallery.html#live-editor-example-10>
2. Basic Plot (2D Line Plot): https://in.mathworks.com/help/matlab/ref/plot.html?#btzitol_sep_mw_3a76f056-2882-44d7-8e73-c695c0c54ca8
3. Line Properties (Customizing the lines):
<https://in.mathworks.com/help/matlab/ref/matlab.graphics.chart.primitive.line-properties.html>